디버깅 도구와 진단 시스템 분석

개념적 배경

왜 디버깅 도구가 필요한가?

디버깅 도구와 진단 시스템은 복잡한 블록체인 시스템에서 발생하는 문제를 식별, 분석, 해결하기 위한 필수적인 도구입니다. geth에서는 다음과 같은 목적으로 사용됩니다:

1. **문제 진단**: 시스템 오류, 성능 저하, 비정상 동작 원인 파악
2. **상태 검사**: 노드 상태, 네트워크 연결, 동기화 상태 확인
3. **로그 분석**: 시스템 이벤트 추적을 통한 문제 해결
4. **대화형 디버깅**: 실시간 명령 실행을 통한 동적 분석
5. **성능 분석**: 병목 지점 식별과 최적화 방향 제시
6. **개발 지원**: 스마트 컨트랙트 개발 및 테스트 지원

다른 접근법과의 비교

| 접근법 | 장점 | 단점 | 사용 사례 |

|--------|------|------|----------|

| **구조화된 로깅** | 체계적, 검색 가능 | 오버헤드 존재 | 프로덕션 모니터링 |

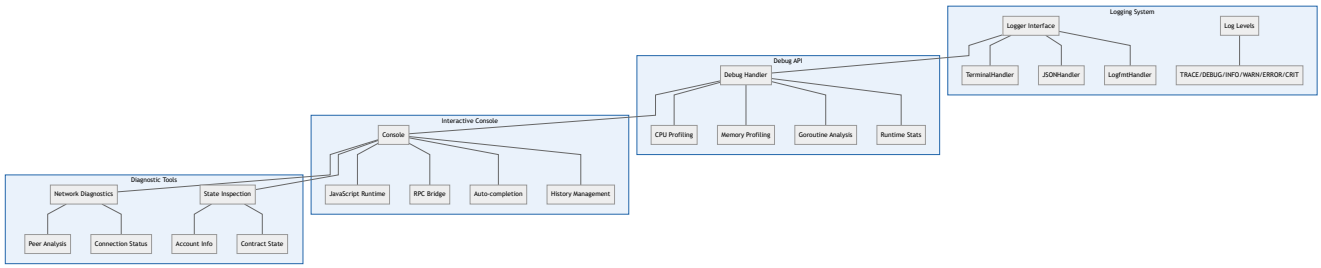
| **대화형 콘솔** | 실시간, 유연성 | 수동 작업 필요 | 개발, 디버깅 |

| **RPC API** | 자동화 가능 | 제한된 기능 | 모니터링 도구 |

| **외부 디버거** | 강력한 기능 | 복잡한 설정 | 심층 분석 |

구현 분석

핵심 아키텍처



주요 컴포넌트

1. 로깅 시스템 (log/)

Logger 인터페이스

```
type Logger interface {
With(ctx ...interface{}) Logger
New(ctx ...interface{}) Logger
```

```
Log(level slog.Level, msg  string, ctx ...interface{})

Trace(msg  string, ctx ...interface{})

Debug(msg  string, ctx ...interface{})

Info(msg  string, ctx ...interface{})

Warn(msg  string, ctx ...interface{})

Error(msg  string, ctx ...interface{})

Crit(msg  string, ctx ...interface{})

Write(level slog.Level, msg  string, attrs ...any)

Enabled(ctx context.Context, level slog.Level) bool

Handler() slog.Handler

}
```

특징:

- 구조화된 로깅 지원
- 컨텍스트 기반 로그 속성
- 레벨별 필터링
- 다양한 출력 형식 지원

로그 레벨 시스템

```
const (

LevelTrace slog.Level = -8  // 가장 상세한 추적 정보

LevelDebug = slog.LevelDebug  // 디버깅 정보

LevelInfo = slog.LevelInfo  // 일반 정보
```

```
LevelWarn = slog.LevelWarn  // 경고

LevelError = slog.LevelError  // 오류

LevelCrit slog.Level = 12  // 치명적 오류 (프로그램 종료)

)


func  FromLegacyLevel(lvl  int) slog.Level {

switch  lvl {

case  legacyLevelCrit:

return  LevelCrit

case  legacyLevelError:

return  slog.LevelError

case  legacyLevelWarn:

return  slog.LevelWarn

case  legacyLevelInfo:

return  slog.LevelInfo

case  legacyLevelDebug:

return  slog.LevelDebug

case  legacyLevelTrace:

return  LevelTrace

}

return  LevelCrit

}
```

터미널 핸들러

```go
type TerminalHandler struct {
mu sync.Mutex
wr io.Writer
lvl slog.Level
useColor  bool
attrs []slog.Attr
fieldPadding  map[string]int  // 필드 패딩 최적화
buf []byte
}

func (h *TerminalHandler) Handle(_ context.Context, r slog.Record) error {
h.mu.Lock()
defer  h.mu.Unlock()
buf := h.format(h.buf, r, h.useColor)
h.wr.Write(buf)
h.buf = buf[:0]
return  nil
}
```

특징:

- 컬러 출력 지원
- 필드 정렬 최적화
- 사람이 읽기 쉬운 형식

- 실시간 출력

로그 포맷팅

```
func (h *TerminalHandler) format(buf []byte, r slog.Record, usecolor  bool) []by

msg := escapeMessage(r.Message)

var  color = ""

if  usecolor {

switch  r.Level {

case  LevelCrit:

color = "\x1b[35m"  // 자홍색

case  slog.LevelError:

color = "\x1b[31m"  // 빨간색

case  slog.LevelWarn:

color = "\x1b[33m"  // 노란색

case  slog.LevelInfo:

color = "\x1b[32m"  // 초록색

case  slog.LevelDebug:

color = "\x1b[36m"  // 청록색

case  LevelTrace:

color = "\x1b[34m"  // 파란색

}

}

b := bytes.NewBuffer(buf)
```

```
// 레벨 출력
if  color != "" {
b.WriteString(color)
b.WriteString(LevelAlignedString(r.Level))
b.WriteString("\x1b[0m")
} else {
b.WriteString(LevelAlignedString(r.Level))
}
// 시간 출력
b.WriteString("[")
writeTimeTermFormat(b, r.Time)
b.WriteString("] ")
b.WriteString(msg)
// 속성 출력
h.formatAttributes(b, r, color)
return  b.Bytes()
}
```

2. 대화형 콘솔 (console/)

콘솔 구조체

```
type Console struct {
client *rpc.Client // RPC 클라이언트
jsre *jsre.JSRE // JavaScript 런타임
```

```
prompt string // 프롬프트 문자열
prompter prompt.UserPrompter // 사용자 입력 처리기
histPath string // 히스토리 파일 경로
history []string // 명령 히스토리
printer io.Writer // 출력 스트림
interactiveStopped chan struct{}
stopInteractiveCh chan struct{}
signalReceived chan struct{}
stopped chan struct{}
wg sync.WaitGroup
stopOnce sync.Once
}
```

특징:

- JavaScript 기반 스크립팅
- 명령 히스토리 관리
- 자동 완성 지원
- 신호 처리 (Ctrl+C)

JavaScript 런타임 초기화

```
func (c *Console) init(preload []string) error {
c.initConsoleObject()
// JavaScript <-> Go RPC 브리지 초기화
```

```
bridge := newBridge(c.client, c.prompter, c.printer)

if err := c.initWeb3(bridge); err != nil {

return err

}

if err := c.initExtensions(); err != nil {

return err

}

// 관리자 API 추가

c.jsre.Do(func(vm *goja.Runtime) {

c.initAdmin(vm, bridge)

})

// JavaScript 파일 사전 로드

for _, path := range preload {

if err := c.jsre.Exec(path); err != nil {

return fmt.Errorf("%s: %v", path, err)

}

}

// 히스토리 및 자동 완성 설정

if c.prompter != nil {

if content, err := os.ReadFile(c.histPath); err != nil {

c.prompter.SetHistory(nil)

} else {

c.history = strings.Split(string(content), "\n")
```

```
c.prompter.SetHistory(c.history)

}

c.prompter.SetWordCompleter(c.AutoCompleteInput)

}

return nil

}
```

자동 완성 시스템

```
func (c *Console) AutoCompleteInput(line string, pos int) (string, []string, s

if len(line) == 0 || pos == 0 {

return "", nil, ""

}

// 자동 완성 대상 부분 추출

start := pos - 1

for ; start > 0; start-- {

c := line[start]

if c == '.' || (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') || (c >= '1' &&

continue

}

start++

break

}

return line[:start], c.jsre.CompleteKeywords(line[start:pos]), line[pos:]
```

```
}
```

RPC 브리지

```
type bridge struct {
client *rpc.Client
prompter prompt.UserPrompter
printer io.Writer
}

func (b *bridge) Send(call jsre.Call) (goja.Value, error) {
// JavaScript 요청을 Go 값으로 변환
reqVal, err := call.Argument(0).ToObject(call.VM).MarshalJSON()
if  err != nil {
return  nil, err
}
var (
rawReq = string(reqVal)
dec = json.NewDecoder(strings.NewReader(rawReq))
reqs []jsonrpcCall
batch  bool
)
// 배치 요청 처리
```

```
if rawReq[0] == '[' {
batch = true
dec.Decode(&reqs)
} else {
batch = false
reqs = make([]jsonrpcCall, 1)
dec.Decode(&reqs[0])
}
// 요청 실행
var resps []*goja.Object
for _, req := range reqs {
resp := call.VM.NewObject()
resp.Set("jsonrpc", "2.0")
resp.Set("id", req.ID)
var result json.RawMessage
if err = b.client.Call(&result, req.Method, req.Params...); err == nil {
if result == nil {
resp.Set("result", goja.Null())
} else {
// JSON 파싱 및 결과 설정
JSON := call.VM.Get("JSON").ToObject(call.VM)
parse, _ := goja.AssertFunction(JSON.Get("parse"))
resultVal, err := parse(goja.Null(), call.VM.ToValue(string(result)))
```

```
if err != nil {
setError(resp, -32603, err.Error(), nil)
} else {
resp.Set("result", resultVal)
}
}
} else {
// 오류 처리
code := -32603
var data interface{}
if rpcErr, ok := err.(rpc.Error); ok {
code = rpcErr.ErrorCode()
}
if dataErr, ok := err.(rpc.DataError); ok {
data = dataErr.ErrorData()
}
setError(resp, code, err.Error(), data)
}
resps = append(resps, resp)
}
// 결과 반환
if batch {
return call.VM.ToValue(resps), nil
```

```
} else {
return resps[0], nil
}
}
```

3. 디버그 API (internal/debug/)

핵심 기능들

```
type HandlerT struct {
mu sync.Mutex
cpuW io.WriteCloser // CPU 프로파일 출력
cpuFile string // CPU 프로파일 파일명
traceW io.WriteCloser // Go trace 출력
traceFile string // Go trace 파일명
}

// 런타임 통계 수집
func (*HandlerT) MemStats() *runtime.MemStats {
s := new(runtime.MemStats)
runtime.ReadMemStats(s)
return s
}

// GC 통계 수집
```

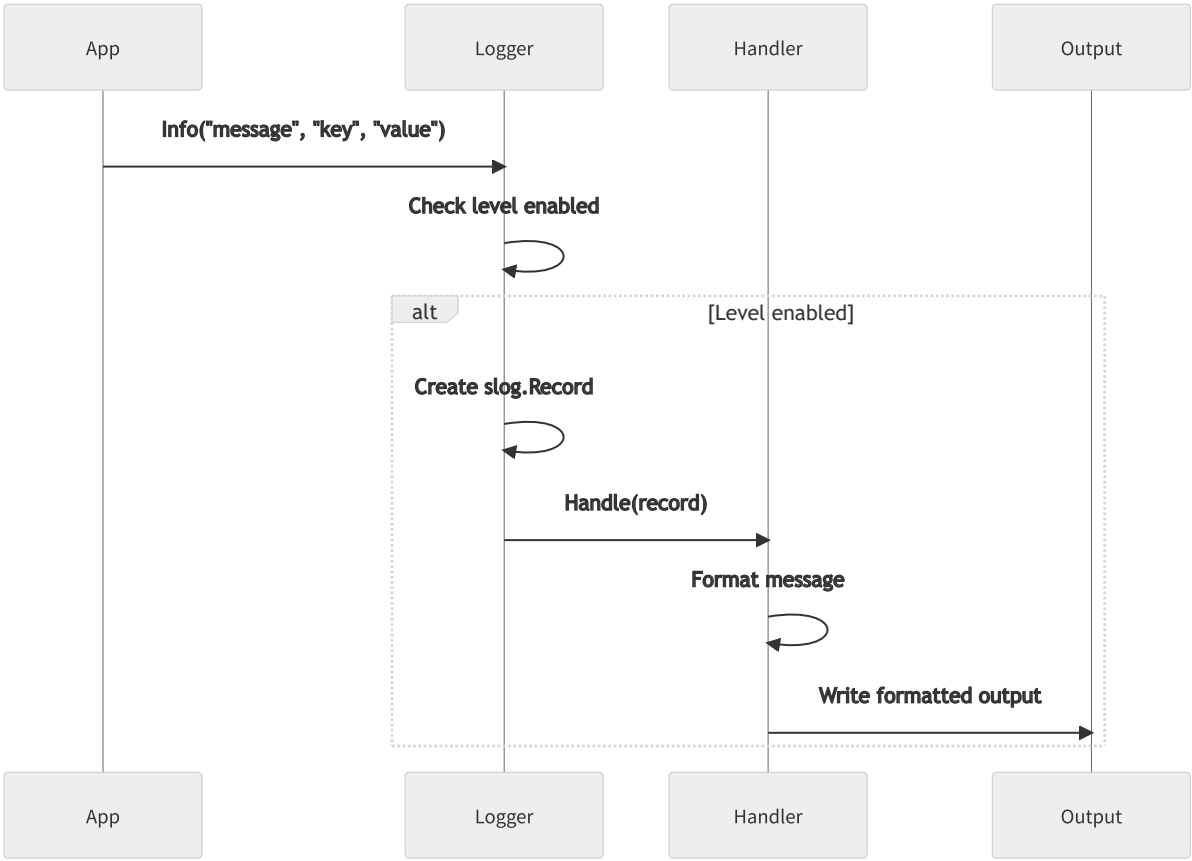
```
func (*HandlerT) GcStats() *debug.GCStats {
s := new(debug.GCStats)
debug.ReadGCStats(s)
return s
}


// 고루틴 스택 덤프
func (*HandlerT) Stacks(filter *string) string {
buf := new(bytes.Buffer)
pprof.Lookup("goroutine").WriteTo(buf, 2)
// 필터링 적용
if filter != nil && len(*filter) > 0 {
// 논리 표현식을 파싱 가능한 형태로 변환
expanded := *filter
expanded = regexp.MustCompile(`[:/\.A-Za-z0-9_-]+`).ReplaceAllString(expanded, "
expanded = regexp.MustCompile("!(`[:/\\.A-Za-z0-9_-]+`)").ReplaceAllString(expan
expanded = strings.ReplaceAll(expanded, "||", "or")
expanded = strings.ReplaceAll(expanded, "&&", "and")
expr, err := bexpr.CreateEvaluator(expanded)
if err != nil {
log.Error("Failed to parse filter expression", "expanded", expanded, "err", err)
return ""
}
```

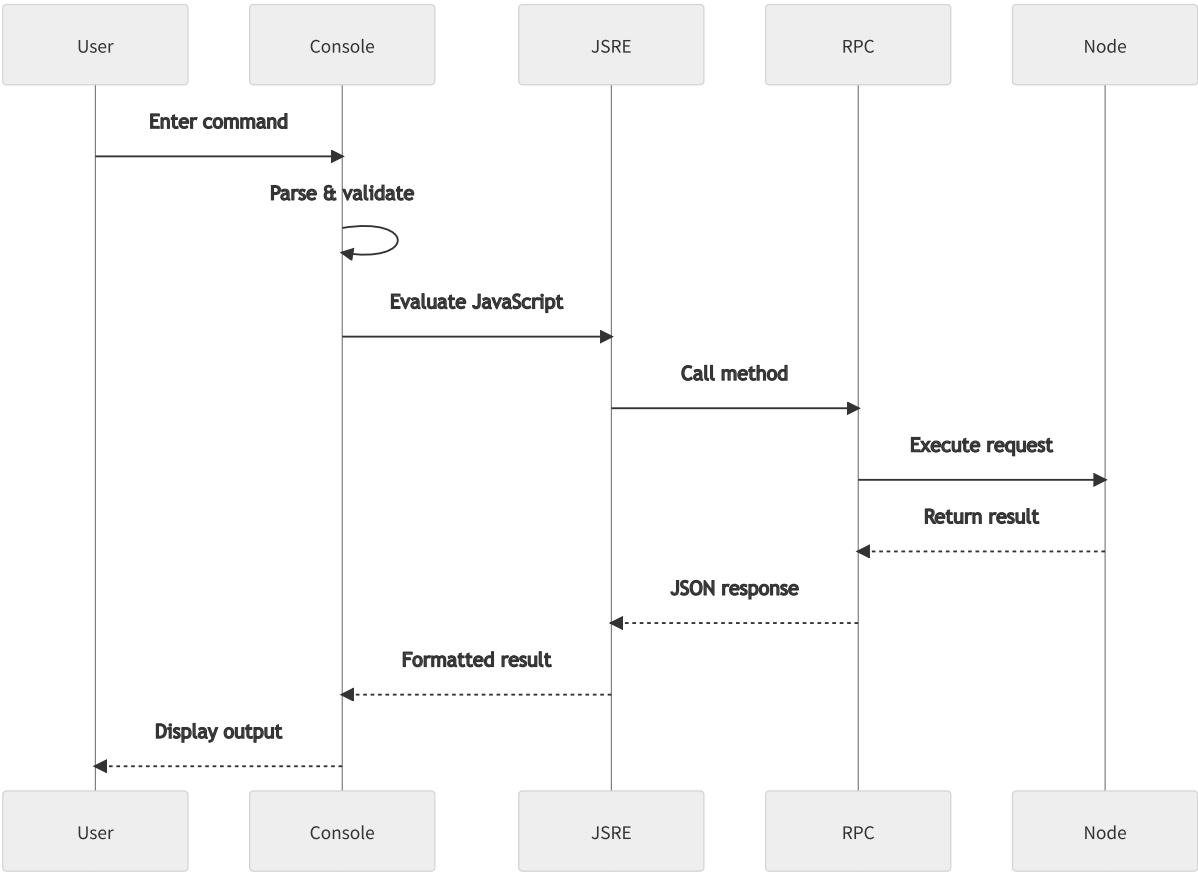
```
// 고루틴 덤프를 세그먼트별로 필터링
dump := buf.String()
buf.Reset()
for _, trace := range strings.Split(dump, "\n\n") {
if ok, _ := expr.Evaluate(map[string]string{"Value": trace}); ok {
buf.WriteString(trace)
buf.WriteString("\n\n")
}
}
}
return buf.String()
}
```

실행 흐름

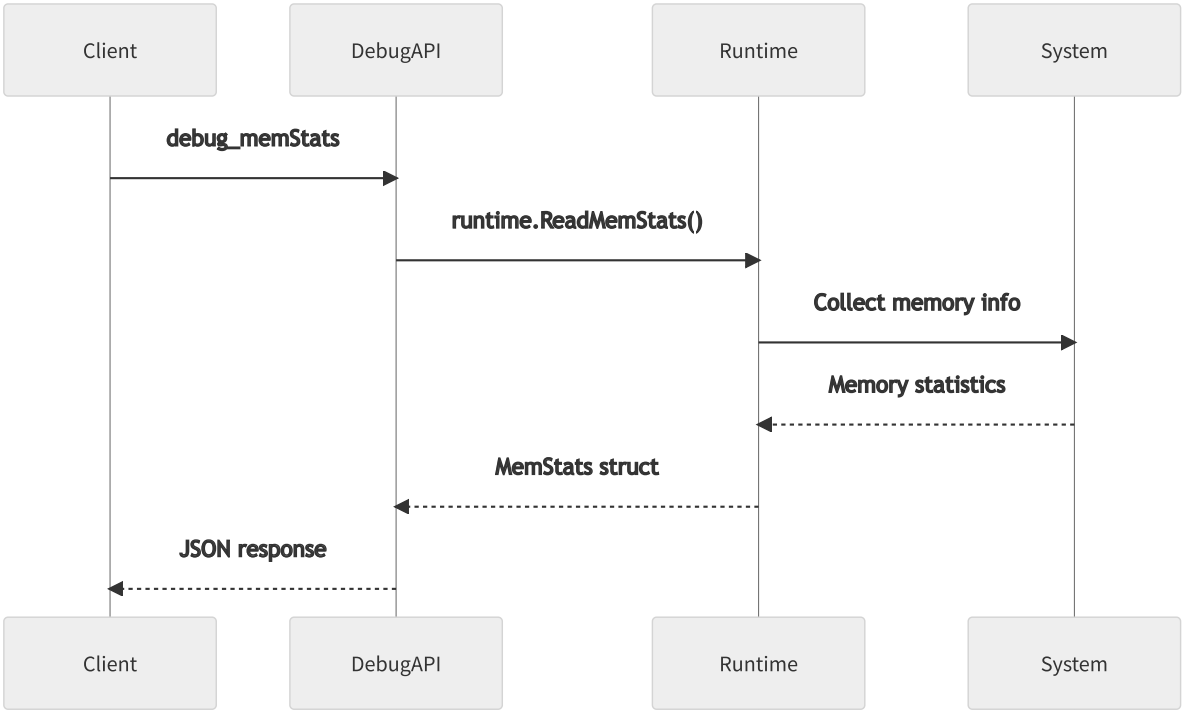
1. 로깅 시스템 흐름


App
Logger
Handler
Output
Info("message", "key", "value")
Check level enabled
alt
[Level enabled]
Create slog.Record
Handle(record)
Format message
Write formatted output
App
Logger
Handler
Output

2. 콘솔 대화형 세션 흐름


User
Console
JSRE
RPC
Node
Enter command
Parse & validate
Evaluate JavaScript
Call method
Execute request
Return result
JSON response
Formatted result
Display output
User
Console
JSRE
RPC
Node

3. 디버그 정보 수집 흐름



로그 분석과 상태 검사 도구

1. 구조화된 로그 분석

로그 형식 예시

```
INFO [12-15|14:30:45.123] Block synchronisation started synced=0 cache=16.00MiB

WARN [12-15|14:30:46.456] Synchronisation failed, retrying err="peer disconnect"

ERROR[12-15|14:30:47.789] Failed to insert block number=12345 hash=0x1234...
```

로그 파싱 도구

```
# 특정 레벨 로그만 추출
grep "ERROR" geth.log

# 시간 범위별 로그 추출
awk '/14:30:45/,/14:30:50/' geth.log

# JSON 형식 로그 분석
jq '.level == "error"' geth.json

# 특정 키워드 통계
grep -c "Block synchronisation" geth.log
```

2. 네트워크 진단 도구

피어 연결 상태 확인

```
// 콘솔에서 실행
admin.peers // 연결된 피어 목록
net.peerCount // 피어 수
admin.nodeInfo // 노드 정보
net.listening // 리스닝 상태

// 특정 피어 정보
admin.peers.forEach(function(peer) {
```

```
console.log("Peer:", peer.network.remoteAddress,
"Protocols:", Object.keys(peer.protocols));
});
```

네트워크 연결 진단

```
// 네트워크 상태 종합 진단
function networkDiagnosis() {
var info = {
nodeInfo: admin.nodeInfo,
peerCount: net.peerCount,
listening: net.listening,
peers: admin.peers.map(function(p) {
return {
id: p.id.substring(0, 16) + "...",
name: p.name,
network: p.network.remoteAddress,
protocols: Object.keys(p.protocols)
};
})
};
console.log("=== Network Diagnosis ===");
console.log("Node ID:", info.nodeInfo.id.substring(0, 16) + "...");
```

```
console.log("Listening:", info.listening);

console.log("Peer Count:", info.peerCount);

console.log("Peers:", JSON.stringify(info.peers, null, 2));

return  info;

}
```

3. 상태 검사 도구

블록체인 상태 확인

```
// 동기화 상태 확인

function  syncStatus() {

var  sync  =  eth.syncing;

if (sync === false) {

console.log("Node is fully synced");

console.log("Current block:", eth.blockNumber);

console.log("Latest block time:", new Date(eth.getBlock("latest").timestamp * 10

} else {

console.log("Syncing in progress:");

console.log("Current block:", sync.currentBlock);

console.log("Highest block:", sync.highestBlock);

console.log("Progress:", (sync.currentBlock / sync.highestBlock * 100).toFixed(2

}

}
```

```
// 계정 상태 확인
function accountStatus(address) {
var balance = eth.getBalance(address);
var nonce = eth.getTransactionCount(address);
var code = eth.getCode(address);
console.log("Address:", address);
console.log("Balance:", web3.fromWei(balance, "ether"), "ETH");
console.log("Nonce:", nonce);
console.log("Contract:", code !== "0x" ? "Yes" : "No");
return {
address: address,
balance: balance,
nonce: nonce,
isContract: code !== "0x"
};
}
```

트랜잭션 풀 상태

```
// 트랜잭션 풀 분석
function txPoolStatus() {
var status = txpool.status;
var content = txpool.content;
```

```
console.log("=== Transaction Pool Status ===");

console.log("Pending transactions:", status.pending);

console.log("Queued transactions:", status.queued);

// 계정별 트랜잭션 수

var accountTxs = {};

Object.keys(content.pending).forEach(function(addr) {

accountTxs[addr] = Object.keys(content.pending[addr]).length;

});

console.log("Transactions by account:");

Object.keys(accountTxs).forEach(function(addr) {

console.log(" ", addr.substring(0, 10) + "...:", accountTxs[addr]);

});

return {

status: status,

accountTxs: accountTxs

};

}
```

문제 해결 가이드

1. 일반적인 문제 진단

동기화 문제

```
// 동기화 문제 진단 스크립트
function diagnoseSyncIssues() {
console.log("=== Sync Diagnosis ===");
// 1. 피어 연결 확인
var peerCount = net.peerCount;
console.log("Connected peers:", peerCount);
if (peerCount < 3) {
console.log("WARNING: Low peer count. Check network connectivity.");
}
// 2. 동기화 상태 확인
var sync = eth.syncing;
if (sync !== false) {
var progress = (sync.currentBlock / sync.highestBlock * 100).toFixed(2);
console.log("Sync progress:", progress + "%");
console.log("Current block:", sync.currentBlock);
console.log("Highest block:", sync.highestBlock);
if (sync.currentBlock === sync.startingBlock) {
console.log("WARNING: Sync appears stuck. Consider restarting.");
}
}
// 3. 최근 블록 시간 확인
var latestBlock = eth.getBlock("latest");
```

```
var blockAge = (Date.now() / 1000) - latestBlock.timestamp;

console.log("Latest block age:", Math.floor(blockAge), "seconds");

if (blockAge > 300) {

console.log("WARNING: Latest block is old. Sync may be behind.");

}

}
```

성능 문제 진단

```
// 성능 문제 진단

function diagnosePerformance() {

console.log("=== Performance Diagnosis ===");

// 1. 메모리 사용량 확인

var memStats = debug.memStats();

console.log("Heap size:", (memStats.HeapSys / 1024 / 1024).toFixed(2), "MB");

console.log("Heap in use:", (memStats.HeapInuse / 1024 / 1024).toFixed(2), "MB")

console.log("GC runs:", memStats.NumGC);

// 2. 고루틴 수 확인

var goroutines = debug.memStats().NumGoroutine;

console.log("Goroutines:", goroutines);

if (goroutines > 1000) {

console.log("WARNING: High goroutine count. Possible goroutine leak.");

}

// 3. 트랜잭션 풀 크기 확인
```

```
var txStatus = txpool.status;
console.log("Pending txs:", txStatus.pending);
console.log("Queued txs:", txStatus.queued);
if (txStatus.pending > 1000) {
console.log("WARNING: Large transaction pool. May affect performance.");
}
}
```

2. 로그 기반 문제 해결

로그 분석 스크립트

```
#!/bin/bash
# geth 로그 분석 스크립트

LOG_FILE=${1:-geth.log}

echo "=== Geth Log Analysis ==="
echo "Log file: $LOG_FILE"
echo

# 오류 통계
echo "Error Statistics:"
echo "CRIT: $(grep -c "CRIT" $LOG_FILE)"
```

```
echo "ERROR: $(grep -c "ERROR" $LOG_FILE)"
echo "WARN: $(grep -c "WARN" $LOG_FILE)"
echo

# 최근 오류들
echo "Recent Errors (last 10):"
grep "ERROR\|CRIT" $LOG_FILE | tail -10
echo

# 동기화 관련 로그
echo "Sync Related Messages:"
grep -i "sync\|import\|download" $LOG_FILE | tail -5
echo

# 네트워크 관련 로그
echo "Network Related Messages:"
grep -i "peer\|connect\|disconnect" $LOG_FILE | tail -5
echo

# 성능 관련 로그
echo "Performance Related Messages:"
grep -i "slow\|timeout\|memory\|cpu" $LOG_FILE | tail -5
```

실시간 로그 모니터링

```
# 실시간 오류 모니터링
tail -f geth.log | grep --color=always "ERROR\|CRIT"

# 특정 패턴 모니터링
tail -f geth.log | grep --color=always -E "(sync|peer|block)"

# JSON 로그 실시간 분석
tail -f geth.json | jq 'select(.level == "error" or .level == "warn")'
```

3. 자동화된 진단 도구

헬스체크 스크립트

```
// 종합 헬스체크 함수
function healthCheck() {
var health = {
timestamp: new Date().toISOString(),
node: {
version: admin.nodeInfo.name,
id: admin.nodeInfo.id.substring(0, 16) + "...",
listening: net.listening
},
```

```
network: {
peerCount: net.peerCount,
syncing: eth.syncing !== false,
blockNumber: eth.blockNumber
},
performance: {
memStats: debug.memStats(),
txPool: txpool.status
},
issues: []
};
// 문제 검사
if (health.network.peerCount < 3) {
health.issues.push("Low peer count: " + health.network.peerCount);
}
if (health.performance.memStats.HeapInuse > 2 * 1024 * 1024 * 1024) {
health.issues.push("High memory usage: " +
(health.performance.memStats.HeapInuse / 1024 / 1024 / 1024).toFixed(2) + "GB");
}
if (health.performance.txPool.pending > 1000) {
health.issues.push("Large transaction pool: " + health.performance.txPool.pendin
}
// 최근 블록 시간 확인
```

```
var latestBlock = eth.getBlock("latest");

var blockAge = (Date.now() / 1000) - latestBlock.timestamp;

if (blockAge > 300) {

health.issues.push("Stale latest block: " + Math.floor(blockAge) + "s old");

}

console.log("=== Health Check Results ===");

console.log("Status:", health.issues.length === 0 ? "HEALTHY" : "ISSUES FOUND");

console.log("Node:", health.node.version);

console.log("Peers:", health.network.peerCount);

console.log("Block:", health.network.blockNumber);

console.log("Memory:", (health.performance.memStats.HeapInuse / 1024 / 1024).toF

if (health.issues.length > 0) {

console.log("Issues:");

health.issues.forEach(function(issue) {

console.log(" -", issue);

});

}

return health;

}
```

실제 사용 예시

1. 개발 환경에서의 디버깅

geth 콘솔 시작

```
# 개발 네트워크에서 콘솔 시작
geth  --dev  --http  --http.api="eth,net,web3,personal,debug,admin,txpool"  cons

# 메인넷에 연결하여 콘솔 시작
geth  --http  --http.api="eth,net,web3,debug,admin"  console

# 원격 노드에 연결
geth  attach  http://localhost:8545
```

기본 디버깅 명령어

```
// 노드 정보 확인
admin.nodeInfo
net.version
eth.blockNumber
eth.syncing

// 계정 관리
personal.listAccounts
eth.accounts
eth.getBalance(eth.accounts[0])
```

```
// 트랜잭션 추적
debug.traceTransaction("0x1234...")
debug.traceBlockByNumber(12345)

// 성능 분석
debug.memStats()
debug.gcStats()
debug.stacks()
```

2. 프로덕션 환경에서의 모니터링

로그 설정

```
# 구조화된 로그 출력
geth  --log.format=json  --log.file=/var/log/geth/geth.json

# 로그 레벨 조정
geth  --verbosity=3  # INFO 레벨

# 특정 모듈 로그 레벨 설정
geth  --log.vmodule="p2p=5,eth=4"
```

모니터링 스크립트

```
#!/bin/bash
# 프로덕션 모니터링 스크립트

GETH_RPC="http://localhost:8545"

# 기본 상태 확인
echo "=== Geth Status Check ==="
echo "Timestamp: $(date)"

# RPC 연결 확인
if ! curl -s -X POST --data '{"jsonrpc":"2.0","method":"net_version","param
echo "ERROR: Cannot connect to Geth RPC"
exit 1
fi

# 동기화 상태 확인
SYNC_STATUS=$(curl -s -X POST --data '{"jsonrpc":"2.0","method":"eth_syncin
if [ "$SYNC_STATUS" != "false" ]; then
echo "WARNING: Node is syncing"
echo "Sync status: $SYNC_STATUS"
else
echo "OK: Node is synced"
```

```
fi

# 피어 수 확인
PEER_COUNT=$(curl -s -X POST --data '{"jsonrpc":"2.0","method":"net_peerCou
echo "Peer count: $PEER_COUNT"
if [ $PEER_COUNT -lt 3 ]; then
echo "WARNING: Low peer count"
fi

# 최신 블록 확인
BLOCK_NUMBER=$(curl -s -X POST --data '{"jsonrpc":"2.0","method":"eth_block
echo "Latest block: $BLOCK_NUMBER"

# 메모리 사용량 확인 (debug API 필요)
if curl -s -X POST --data '{"jsonrpc":"2.0","method":"debug_memStats","par
HEAP_SIZE=$(curl -s -X POST --data '{"jsonrpc":"2.0","method":"debug_memSta
echo "Heap usage: $((HEAP_SIZE / 1024 / 1024)) MB"
fi

echo "=== End Status Check ==="
```

3. 문제 해결 시나리오

시나리오 1: 동기화 중단

```
// 1. 동기화 상태 확인
eth.syncing

// 2. 피어 연결 확인
net.peerCount
admin.peers

// 3. 네트워크 연결 테스트
admin.addPeer("enode://...")

// 4. 로그 확인
debug.verbosity(4) // 디버그 레벨 증가

// 5. 강제 동기화 재시작 (주의: 데이터 손실 가능)
// geth removedb 후 재시작
```

시나리오 2: 높은 메모리 사용량

```
// 1. 메모리 통계 확인
var mem = debug.memStats();
console.log("Heap:", mem.HeapInuse / 1024 / 1024, "MB");
console.log("GC runs:", mem.NumGC);
```

```
// 2. 고루틴 수 확인
console.log("Goroutines:", mem.NumGoroutine);

// 3. 강제 GC 실행
debug.freeOSMemory();

// 4. 고루틴 스택 덤프
debug.stacks();

// 5. 메모리 프로파일 생성
debug.writeMemProfile("/tmp/mem.prof");
```

시나리오 3: 트랜잭션 처리 지연

```
// 1. 트랜잭션 풀 상태 확인
txpool.status
txpool.inspect

// 2. 가스 가격 확인
eth.gasPrice
eth.feeHistory(10, "latest", [25, 50, 75])

// 3. 특정 트랜잭션 추적
```

```
debug.traceTransaction("0x...")


// 4. 블록 가스 사용량 확인
eth.getBlock("latest").gasUsed
eth.getBlock("latest").gasLimit
```

관련 시스템

의존성 관계

1. **Go 런타임**: 로깅, 디버깅 기능 제공
2. **JavaScript 엔진**: 콘솔 스크립팅 지원
3. **RPC 시스템**: 원격 디버깅 인터페이스
4. **파일 시스템**: 로그 파일, 히스토리 저장

상호작용 패턴

- **Application → Logger**: 로그 메시지 생성
- **Logger → Handler**: 로그 포맷팅 및 출력
- **Console → RPC**: 명령 실행 요청
- **Debug API → Runtime**: 시스템 정보 수집
- **User → Console**: 대화형 명령 입력

영향 범위

- **성능**: 로깅 레벨에 따른 I/O 오버헤드
- **보안**: 디버그 API 노출 시 보안 위험

- **사용성**: 개발자 생산성 향상
- **운영**: 문제 진단 및 해결 시간 단축

이 디버깅 도구와 진단 시스템은 geth의 개발, 운영, 문제 해결에 필수적인 인프라를 제공하며, 구조화된 로깅, 대화형 콘솔, 다양한 진단 API를 통해 포괄적인 디버깅 환경을 구축합니다.